

Constructors In Depth

Page No. :-

Date :- / /

- It is used to create an instance.
- It's similar to method except:
 - Name: Constructor name is same as class Name.
 - Return type: Constructor do not have any return type.
 - Constructor can not be static or final or abstract, synchronized.

Types of Constructors

- | | |
|----------------|---|
| - Default | Constructor Overloaded |
| - No Args | Private constructor |
| - Parametrized | Constructor chaining <ul style="list-style-type: none">• this• Super |

frequently asked interview question on Constructors

- ① - Why constructor do not have any return type?
- ② - Why constructor can not be final?
- ③ - Why constructor cannot be abstract?
- ④ - Why constructor cannot be static?
- ⑤ - Can we define constructor in interface?
- ⑥ - Why constructor name is same as of class Name?

Employee obj = new Employee()

↳ It's tells to java at runtime, Hey we need to call constructor, Not method.
Obj reference come from new not by constructor

Ans → It is easy to identify a constructor while looking the code

- ① Ans → The constructor does not have a return type because its purpose is to initialize an object, not to return a value.

Ex: class Employee {

No return type
 at runtime implicitly compile as return type
 Employee() { }] → Constructor

Employee / int Employee() { }] → Method
 Having something as return type

```

}
class Main {
    psum() {
        Employee obj = new Employee();
    }
}
  
```

(2) Ans → As we know that final method cannot be overridden, by default we cannot override constructor then there is no need to explicitly add final before constructor, if we declare a parent constructor in child class then it act as method, and method have return type so parent constructor must return something that's why we said constructor cannot be overridden.

Ex: class Manager extends Employee {
 }
 Employee() { } X Not Possible
 int Employee() { } ✓ Possible bec it's a method
 Manager() { } // it's a const of Manager class

Note: Constructor cannot be inherited
 Therefore declaring a const. as final would have no meaning and would have no effect

③ Ans → Since constructor is not inherited and we know that abstract method definition must be provided by child class via inheritance so child class doesn't have access to constructor, and if child class doesn't have access to const. then how does it provide its implementation, that's why constructor cannot be abstracted.

④ Ans → As we know that constructor is used to initialize the instance variables of the class.

But the static keyword definition says that it is used to access static variable and static method only.

point is if we declare a constructor as static then we cannot initialize instance variable.

if we cannot initialize instance variable. then ~~the~~ need of constructor is ended that's why we cannot declare constructor as static.

Another reason is ^{object} constructor chaining (super()).

⑤ → Constructor is used to create an object. As from the definition of interface we cannot create object of interface, then there is no meaning of having const. in interface.

Types of Constructor

- i) **Default Constructor** → A default constructor is a constructor automatically provided by the Java compiler. If no constructor is defined in the class it has no parameters and initializes instance variable to their default values.

```
ex: class Mohit {  
    }  
}
```

On ~~com~~ terminal

Javac Mohit.java → Mohit.class file generated
when we open Mohit.class we can see java automatically added default constructor.

```
class Mohit {  
    Mohit() { }  
}
```

- ii) **No-Argument Constructor** : A noArgs constructor is explicitly written by the programmer that takes no parameters. It is used to initialize objects with default or custom values.

```
class Student {  
    Student() {  
        Sout("Generated by Programmer");  
    }  
}
```

Note : If u define any constructor on your own then compiler doesn't add default constructor at runtime, due to this we have to define ~~default~~ ^{No args} constructor.
default const only added if you don't define any const on your own so keep this in mind.

3. What is parameterized constructor?

→ A parameterized const. accepts parameters to initialize an object with user-provided values at the time of object creation.

```
class Student {
    String name;
    Student(String name) {
        this.name = name;
    }
}
```

4. What is constructor overloading?

→ Constructor overloading means defining multiple constructors in the same class with different parameter lists. It provides multiple ways to create object.

```
class Student {
    Student() {}
    Student(String name) {}
    Student(String name, int age) {}
}
```

5. What is private constructor?

→ A private constructor cannot be accessed from outside the class. It is mainly used to:

- Implement the Singleton design pattern
- Prevent object creation
- Create utility classes with only static methods.

```
class Singleton {
    private Singleton() {}
}
```


6. What is constructor chaining?

→ Constructor chaining is the process of calling one constructor from another constructor to reuse initialization code.

- within the same class use `this()`;
- from the parent class : use `super()`.

Ex. of Singleton class

```
Public class Calculations {
```

```
    String name;
```

```
    int id;
```

```
    private Calculations () { }
```

```
    public static Calculations getInstance () {
        return new Calculations();
    }
```

```
}
```

```
}
```

```
Public class Main {
```

```
    psum () {
```

```
        // Calculations obj = new Calculations();
```

```
        Calculations obj = Calculations.getInstance();
```

```
    }
```

```
}
```

End

Example of this()

```
public class Calculation {
```

```
    String name;
```

```
    int id;
```

```
    Calculation() {
```

```
        this(10);
```

```
    }
    Calculation(int id) {
```

```
        this(id, "Mohit");
```

```
    }
    Calculation(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```

```
}
```

Ex of Super class

```
→ class Person {
```

```
    int id;
```

```
    Person(int id) {
```

```
        this.id = id;
```

```
        System.out.println("Parent constructor: Hi");
```

```
    }
```

```
}
```

```
class Manger extends Person {
```

```
    int age;
```

```
    Manger(int id, int age) {
```

```
        super(id);
```

```
        this.age = age;
```

```
        System.out.println("I am manger: OK dude");
```

```
    }
```

```
}
```

But Parent() call
child() call
afterward
as shown in output.

this phenomenon
is called
constructor
chaining.

calls

this

calls

this one

```
class Main {
```

```
    Psum() {
```

```
        Manger obj = new
```

```
        Manger(101, 20);
```

```
        System.out.println("Parent constructor: Hi");
```

```
        System.out.println("I am manger: OK dude");
```